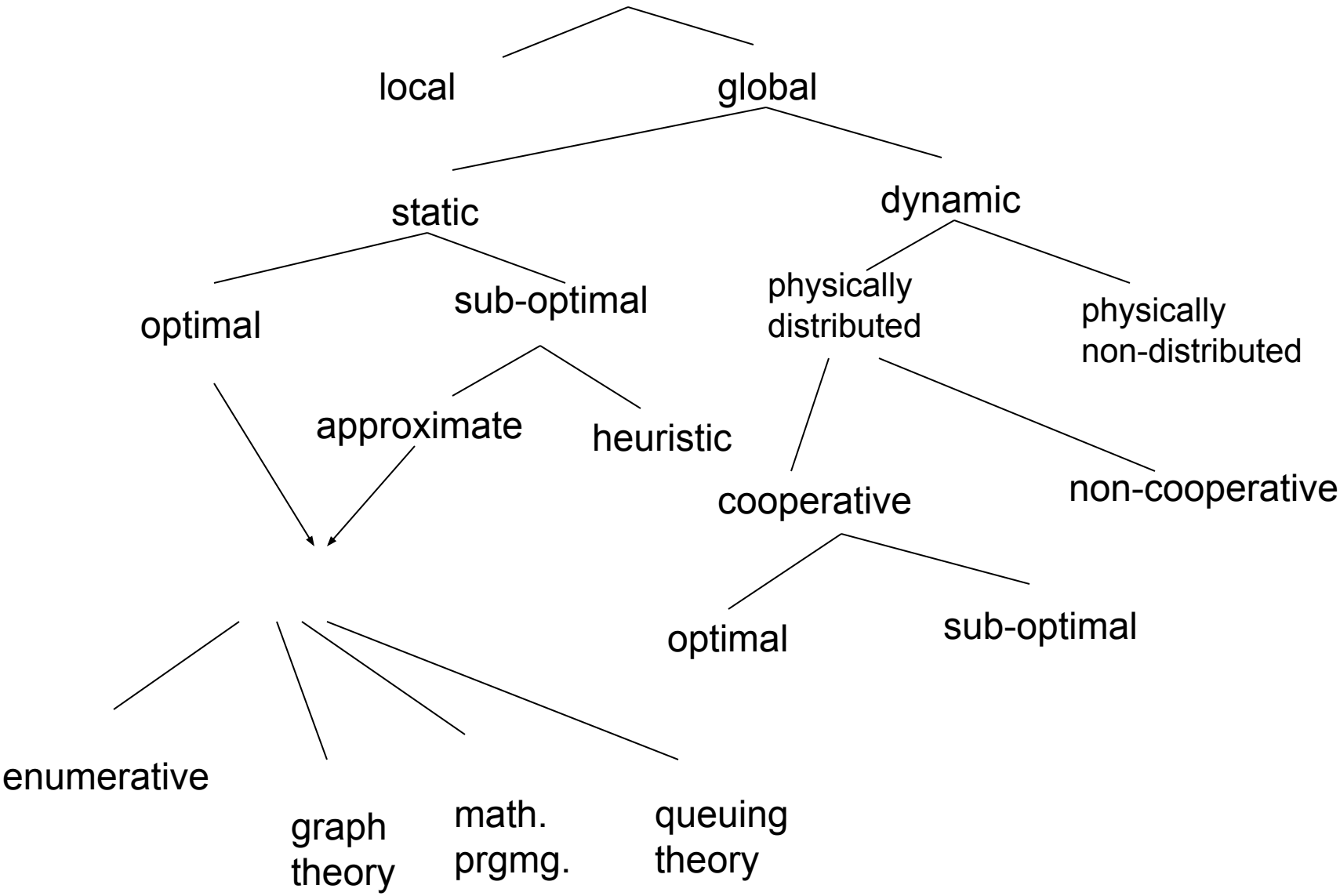
Overview

- Basic Concepts
- A System Performance Model
- Static process Scheduling
- Dynamic Process Scheduling
  - Load Sharing and Load Balancing
- Distributed Process Implementation

Basic Concepts

- Resource allocation: Matching process between the jobs and the resources in order to have:
  - Better performance
  - Load balancing
  - At cheaper cost
- Scheduling (One of the main components in RA)
  - Allocation of processes to processors that resides on the same or different machine (processor allocation)
- A set of processes and rules that regulate them

Task Scheduling Characteristics



Basic Concepts

- Local vs global scheduling
  - Assignment of processes to time-slices of the processor
  - Process of deciding where to execute a process in a multicomputer system (multiprocessor, NOWs etc).
    - Allow autonomy of processors
- distributed vs centralized
  - Scheduling may be carried out by a single authority (at least the major part of it), ease of control
  - All processing elements participate in the distributed nodes, responsiblities and authorities distributed, communication overhead
- cooperative vs work alone

Static vs Dynamic

- Indicate the time at which the scheduling or assignment decision are made
- Static: information about processes and all the subtasks is assumed to be available by the time before doing the scheduling.
  - Task assignments prior to execution
  - Deterministic scheduling or task scheduling.
- Dynamic: Make the decision on the spot with little *a priori* knowledge about the jobs.
  - Task assignments during run-time

Active learning

- Work in group of 4:
- In what condition are suitable to use static or dynamic?
- List out examples of static use cases and dynamic use cases.

Optimal vs Suboptimal

- Information about the state of the system are known, an optimal assignment can be made based on some criterion function.
  - Minimizing total process completion time
  - Maximizing utilization of resources
  - Maximizing system throughput
- Infeasible for optimal, then suboptimal solutions should be tried

Active learning

- Work in pair, give an example of optimal use case and one example of suboptimal use case.

Approximate vs Heuristic

- Approximate: Not searching the entire solution space, we find a “good” one.
  - Availability of a function to evaluate a solution
  - Time required to evaluate a function
  - Some metric value of an optimal solution
  - Intelligently pruning the solution space
- Heuristic: Static algorithms with realistic assumptions about *a priori* knowledge
  - Parameters that affect the system indirect way

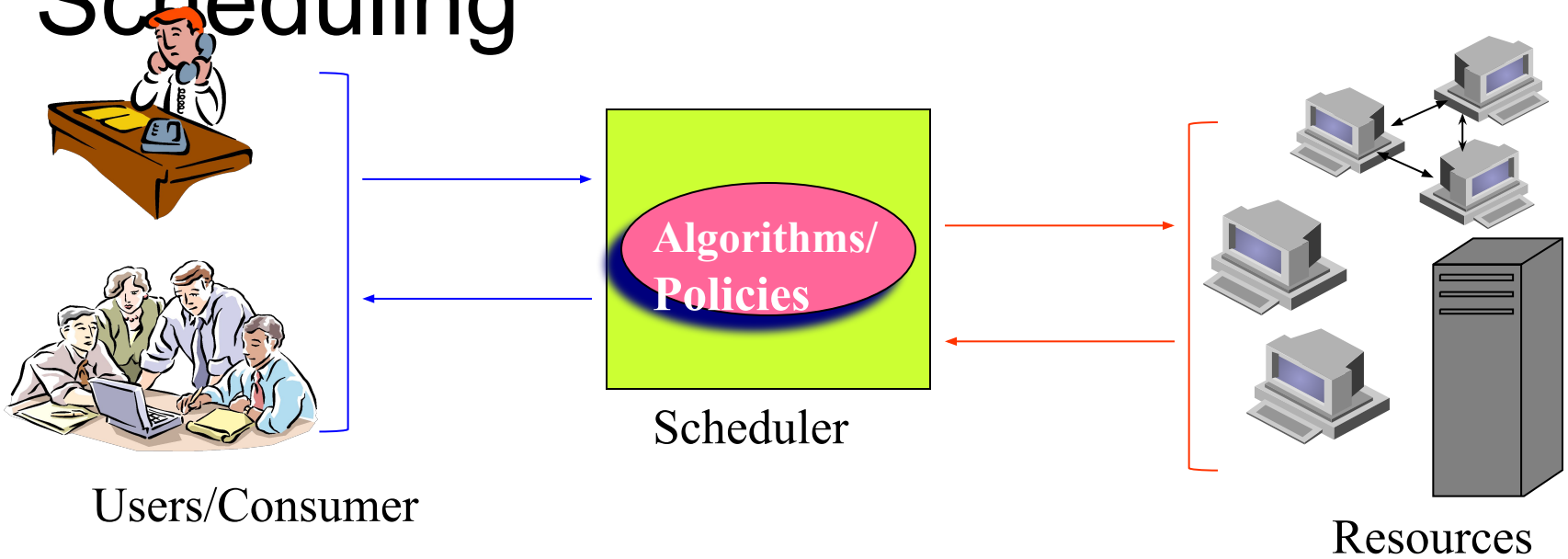
Active learning

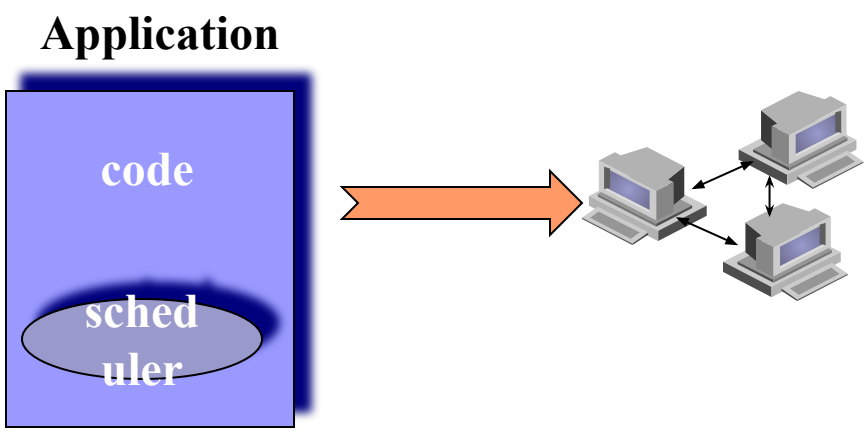
- Work in pair, give an example of heuristic algorithms.
- Briefly describe how it works.

Basic Concepts

- Process-to-processor mapping exists within an algorithm or by itself
  - System level
    - schedule process/jobs
    - Independent module – scheduler
    - More common in distributed systems
  - Application level
    - schedule tasks/data and more specific
    - Usually part of the program (parallel application)
    - Common for parallel applications

Application vs System Level Scheduling





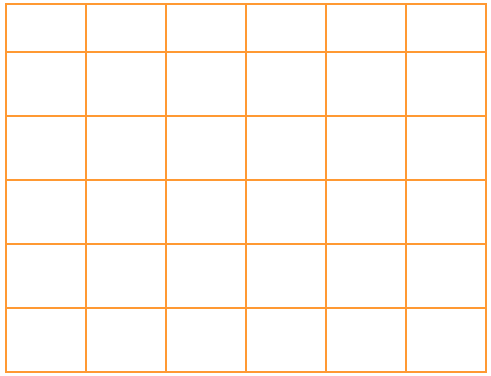
Basic Concepts (cont.)

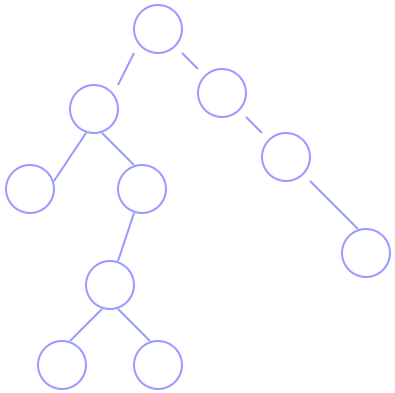
- Metrics
  - Completion time (makespan) , turnaround time
  - Processor utilisation
- Speedup

$$S = \frac{T_{seq}}{T_{par}}$$

Static scheduling

N x N matrix

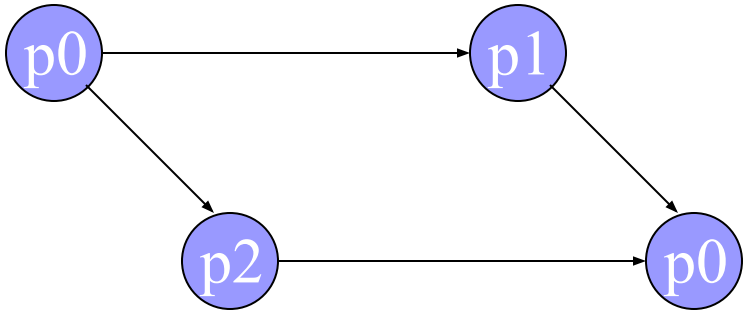




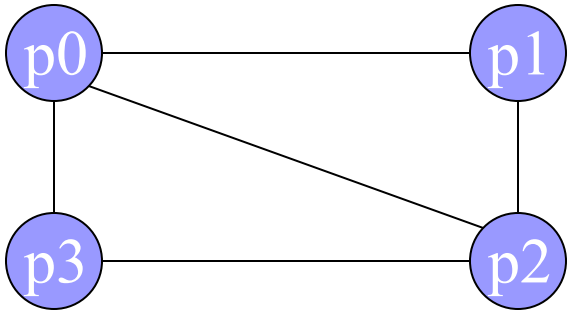
Tree-based application (searching, branch&bound)

A System Performance Model

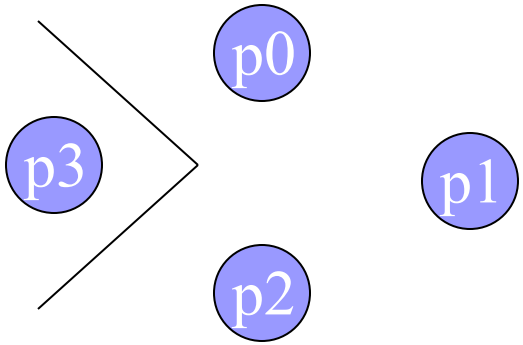
- Process interactions
  - Precedence, Communication, Disjoint model



(a) Precedence



(b) Communication

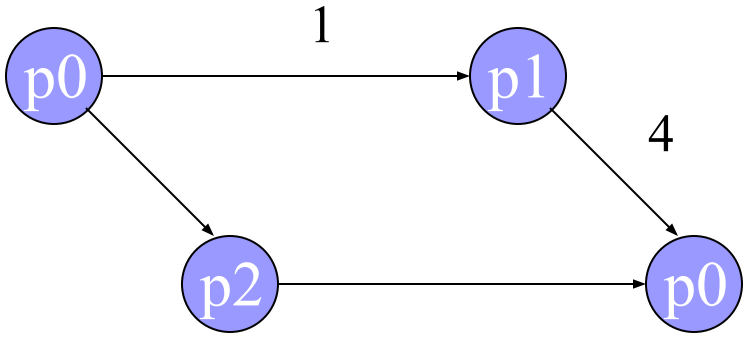


c) Disjoint

4 processes to 2 processors mapping

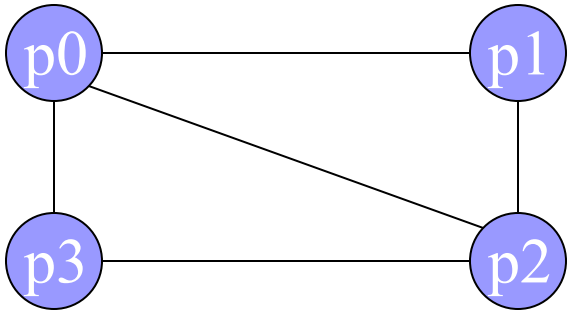
A System Performance Model

- Precedence Process Model
  - Directed Acyclic Graph (DAG)
    - Node – tasks with execution time and Edges – precedence relationship, and labeled with a weight (message units)
    - Wrong process-to-processor mapping may incur communication overhead
  - Best applied for concurrent processes generated by concurrent language construct
  - Objective is to minimise overall computation time (makespan)



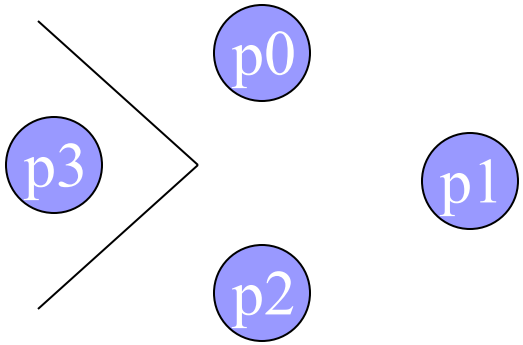
A System Performance Model

- Communication Process Model
  - Undirected graph
  - Execution is not explicit, communication is asynchronous
  - Objective is to minimise the communication overhead



A System Performance Model

- Disjoint process model
  - Ignore dependency issues
- Objective
  - Maximise utilisation – no idle processor
    - Measures - throughput, completion time
  - Fairness
- Benefit must outweigh preemption and migration communication overhead



Precedence Process and Communication System Models

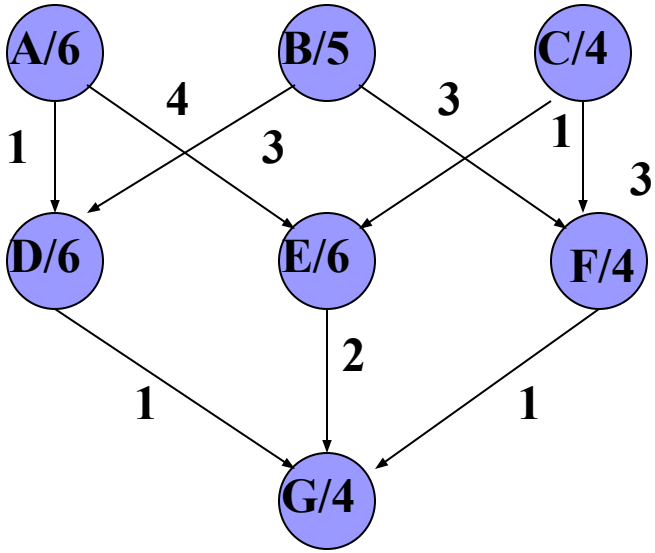
- List scheduling – a simple greedy heuristic algorithm
  - no processor remains idle if there exists tasks it can process.
- Critical path – the longest execution path in a DAG
  - Use it to identify tasks priorities (which to execute first)
- Map all tasks in critical path onto one processor to minimize communication

Precedence Process Model

List scheduling

P1	A/6		D/6		G/4	
P2	B/5		F/4	7		
P3	C/4	2	E/6		4	

Makespan=16



(a) Precedence process model

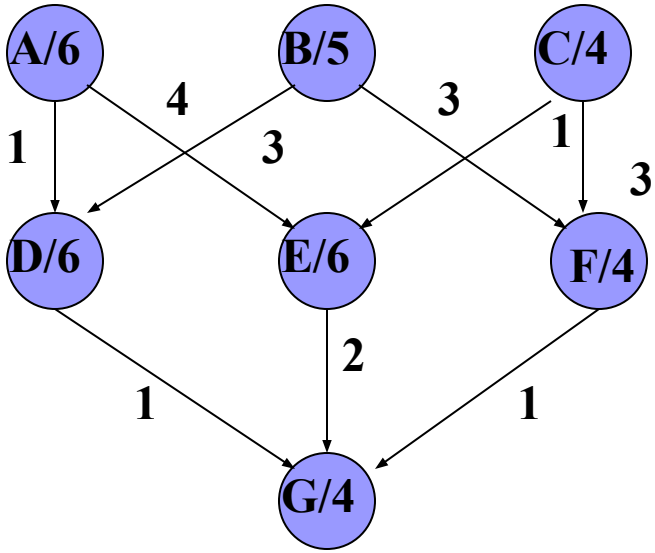
Critical path = ADG or EAG
= 6 + 6 + 4

Precedence Process and Communication System Models

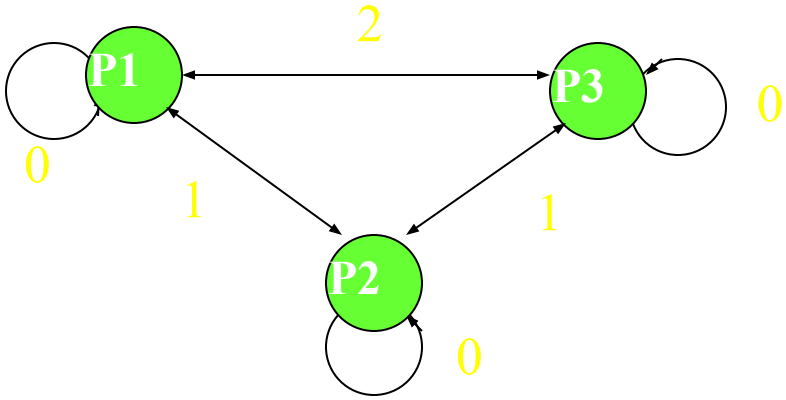
Extended List scheduling

Makespan=28

P1	A/6	2	D/6	10	G/4
P2	B/5	2	F/4	17	
P3	C/4	10	E/6	8	



(a) Precedence process model



(b) Communication system model

Static Process Scheduling

- ***A priori*** knowledge of execution times and communication behaviors of processes.
- Process assignment does not change until execution complete
  - A task is executed on the same processor (processor non-preemptive).
- Goal is to minimize the overall makespan
- Advantages
  - Less overhead because incurred at compile time – more efficient compared to dynamic approach.

Dynamic Process Scheduling

- Redistribution of processes among processors during execution time.
  - Tasks transfer from heavily loaded to lightly loaded processors
  - To achieve load sharing or balancing
- Load balancing operations may be centralised in a single processor or distributed among all the processing elements.
- Advantages & disadvantages
  - Flexible - the system need not be aware of the run-time behaviour of the the applications before execution
  - Run-time overhead

Load Sharing / Balancing

- Load Sharing vs Load Balancing
  - Objective of LB: To balance the load
  - Load Balancing: Reassignment, Dynamic
- Rules that governs a DLB algorithm
  - Information policy
  - Transfer policy
  - Location or placement policy

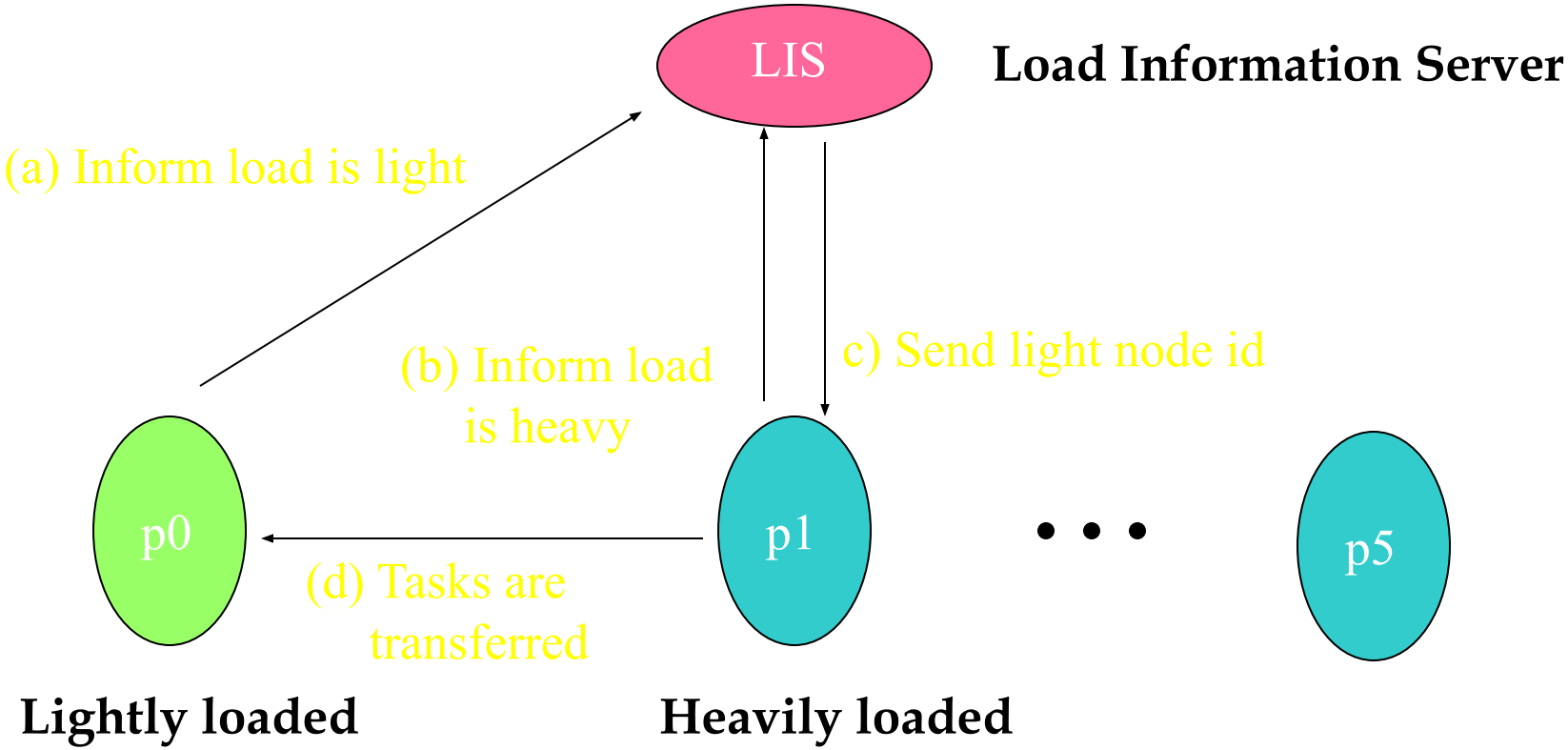
DLB Policies

- Information policy
  - What information to be collected, when from where and how (no load information – e.g. Random, queue length, local vs global)
- Transfer policy
  - Condition for migration/transfer to take place and whether the processor is sender or receiver (threshold, average load between neighbor)
- Location or placement policy
  - Determine the ‘transfer partner’, e.g. which machine to transfer to? Neigbours or anywhere?

Centralised Algorithm: Loadserver

- An example of a centralised algorithm (NOT a hybrid)
- One dedicated process acts as the Load Information Server (LIS)
- All lightly loaded nodes register to LIS
- Heavily loaded nodes request for the id of a lightly loaded node and offload some of its tasks to the lightly loaded nodes
- Policies
  - Information – global knowledge, one dedicated process keeps lightly loaded node id.
  - Transfer – threshold based, sender-initiated
  - Location – anywhere in the system

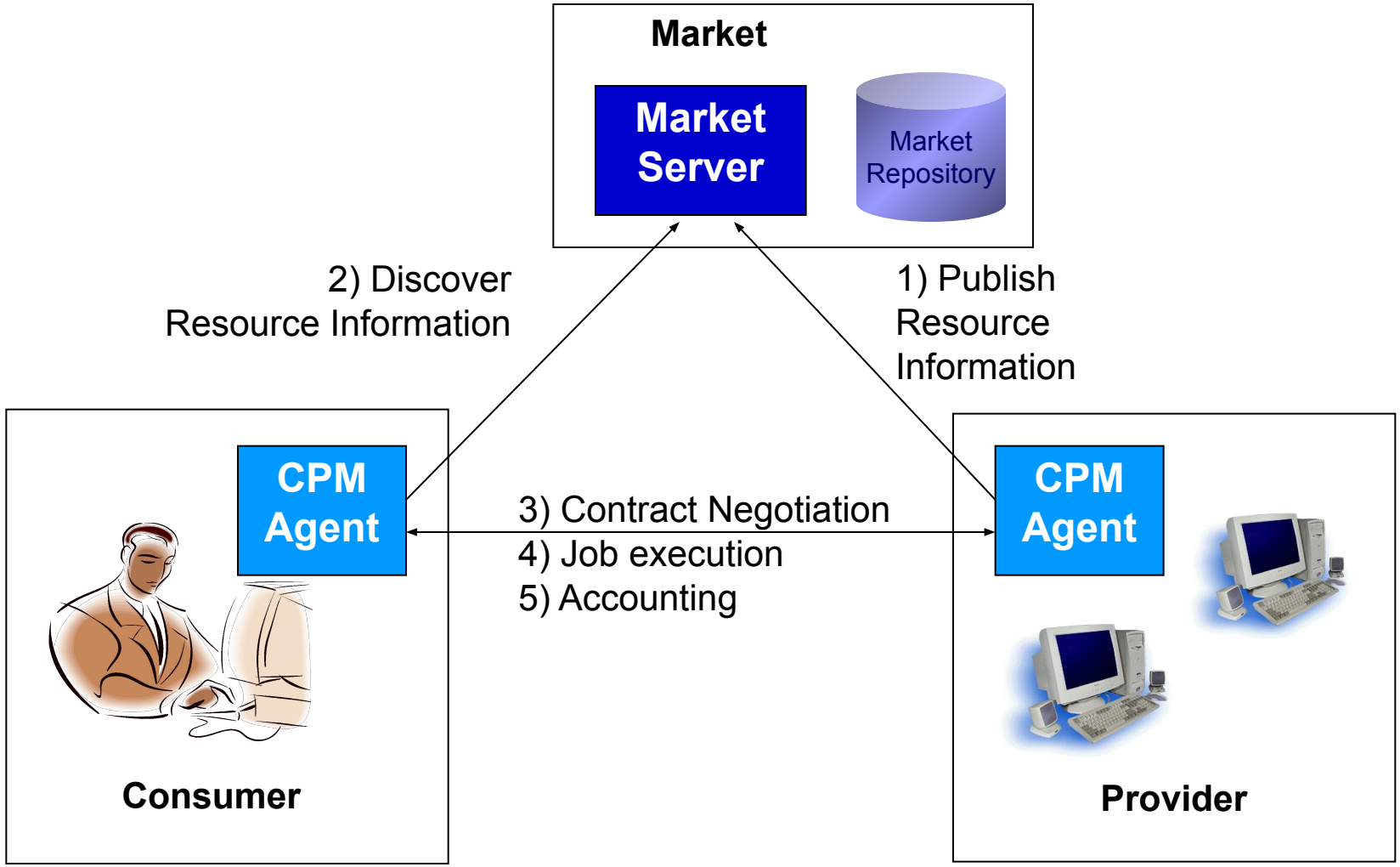
Loadserver Algorithm



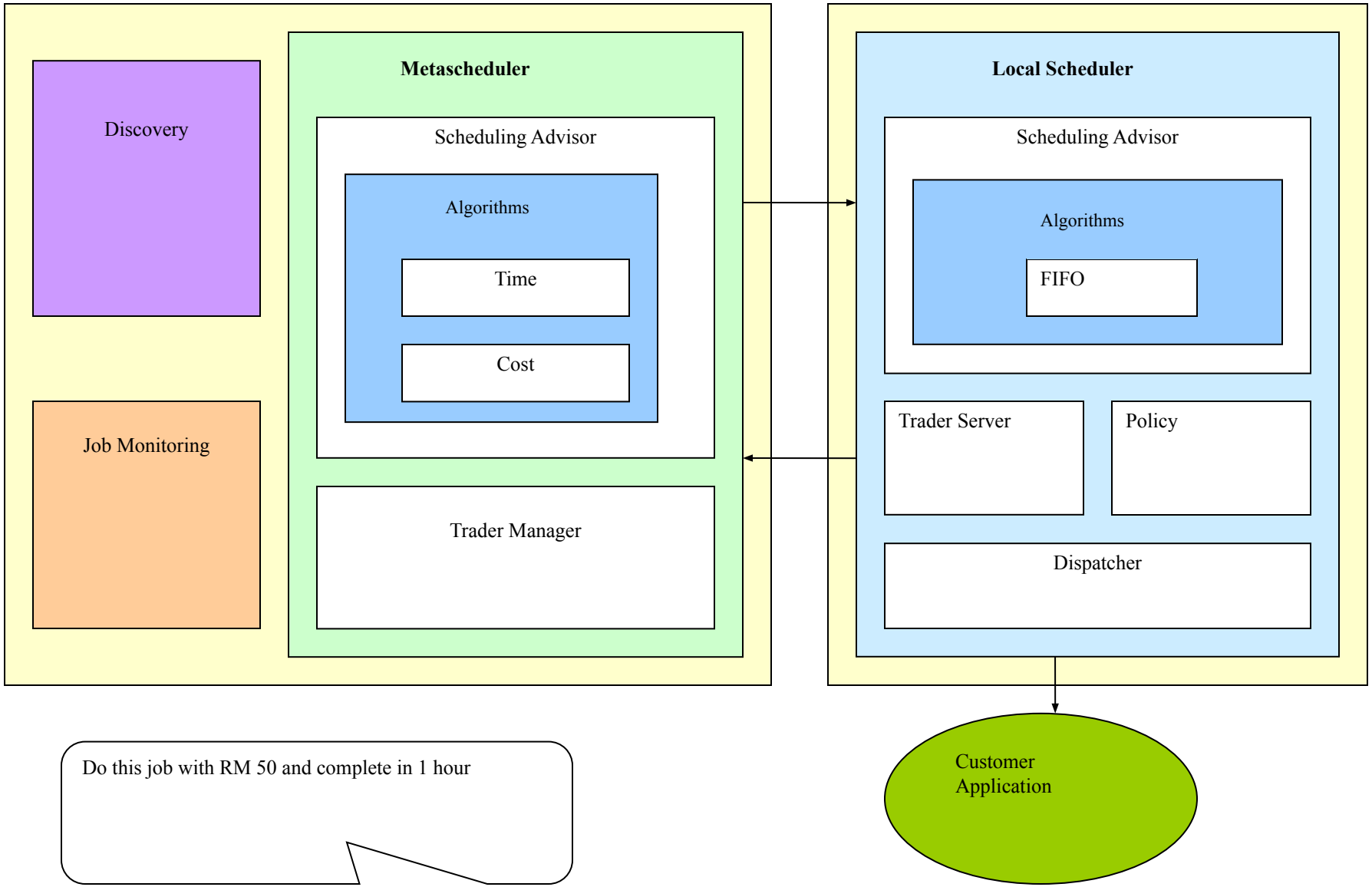
Bidding

- Manager & contractor
- Task announcement by manager, receive bids from contractors (nodes).
- Autonomous in accepting task and delegating task. Winner with the best bid.
- Exchange information in terms of task and bid.
- Economic Framework.

Compute Power Market Interaction Model



Scheduler Architecture

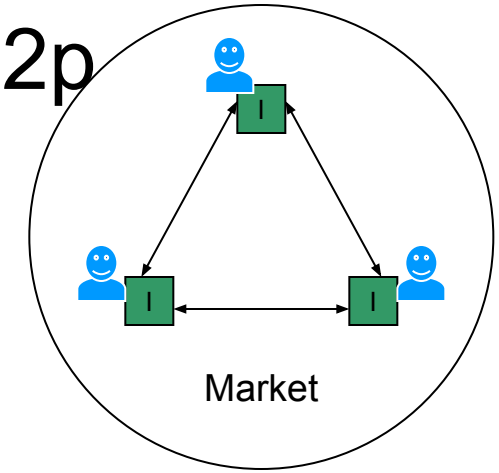


Conceptual Model of e-Science grid

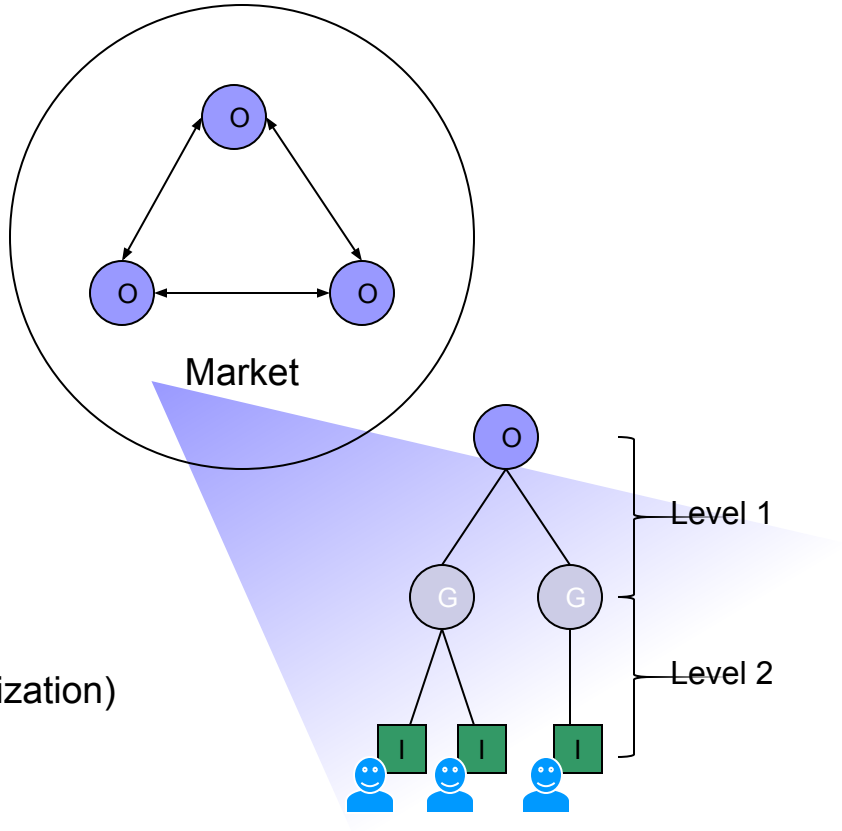
- Service-oriented grid
  - Provider – organization provides services to the grid
- Economic-based grid
  - Based on the compute power market
    - Contract – agreement on services usage between provider and consumer
    - Marketplace – intermediate/ communication medium for both parties (organizations)

P2P vs Grid

CPM/p2p



E-Science GRID

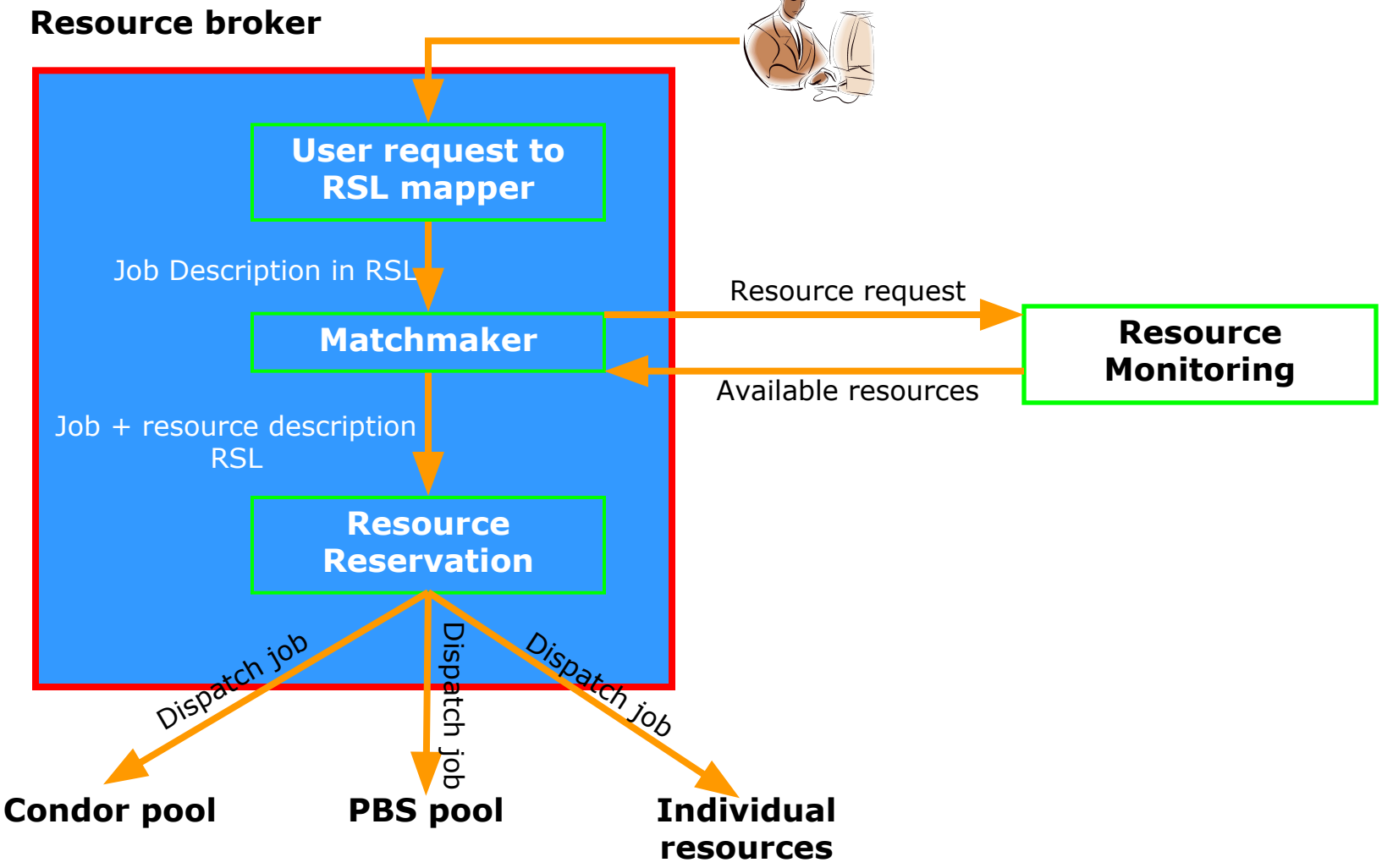


O = Organization, etc USM, UKM, UTM

G = Group (group by Individuals in an organization)

I = Individual user, etc: grid007.cs.usm.my

Resource Broker Framework



Thank you.

Static vs. dynamic scheduling

- Assumptions (for now):
  - 1 instruction issue / cycle
  - Several pipelines with a common IF and ID
    - Ideal CPI still 1, but real CPI won't be 1 but will be closer to 1 than before
    - Same techniques will be used when we look at multiple issue
- Static scheduling (optimized by compiler)
  - When there is a stall (hazard) no further issue of instructions
  - Of course, the stall has to be enforced by the hardware
- Dynamic scheduling (enforced by hardware)
  - Instructions following the one that stalls can issue if they do not produce structural hazards or dependencies

Dynamic scheduling

- Implies possibility of:
  - Out of order issue (we say that an instruction is issued once it has passed the ID stage) and hence out of order execution
  - Out of order completion (also possible in static scheduling but less frequent)
  - Imprecise exceptions (will take care of it later)
- Example (different pipes for add/sub and divide)

R1 = R2/ R3	(long latency)
R2 = R1 + R5	(stall, no issue, because of RAW on R1)
R6 = R7 - R8	(can be issued, executed and completed before the other 2)

Issue and Dispatch

- Split the ID stage into:
  - *Issue* : decode instructions; check for structural hazards and maybe more hazards such as WAW depending on implementations. Stall if there are any. Instructions pass in this stage in order
  - *Dispatch*: wait until no data hazards then read operands. At the next cycle a functional unit, i.e. EX of a pipe, can start executing
- Example revisited.

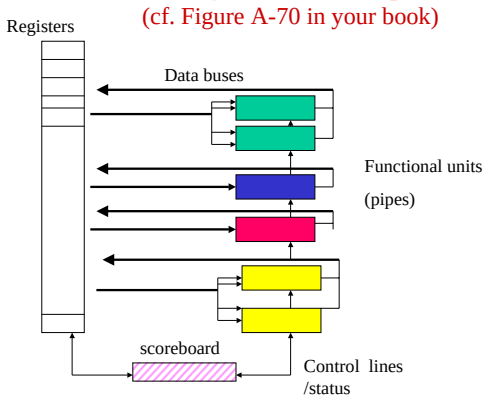
R1 = R2/ R3	(long latency; in execution)
R2 = R1 + R5	(issue but no dispatch because of RAW on R1)
R6 = R7 - R8	(can be issued, dispatched, executed and completed before the other 2)

Implementations of dynamic scheduling

- In order to compute correct results, need to keep track of :
  - execution unit (free or busy)
  - register usage for read and write
  - completion etc.
- Two major techniques
  - Scoreboard (invented by Seymour Cray for the CDC 6600 in 1964)
  - Tomasulo's algorithm (used in the IBM 360/91 in 1967)

Scoreboarding -- The example machine

(cf. Figure A-70 in your book)



Scoreboard basic idea

- The scoreboard keeps a record of all data dependencies
  - Keeps track of which registers are used as sources and destinations and which functional units use them
- The scoreboard keeps a record of all pipe occupancies
  - The original CDC 6600 was not pipelined but conceptually the scoreboard does not depend on pipelining
- The scoreboard decides if an instruction can be issued
  - Either the first time it sees it (no hazard) or, if not, at every cycle thereafter
- The scoreboard decides if an instruction can store its result
  - This is to prevent WAR hazards

An instruction goes through 5 steps

- We assume that the instruction has been successfully fetched (no I-cache miss)
- 1. Issue
  - The execution unit for that instruction type must be *free* (no structural hazard)
  - There should be **no** WAW hazard
  - If either of these conditions is false the instruction stalls. No further issue is allowed
    - There can be more fetches if there is an instruction fetch buffer (like there was in the CDC 6660)

Execution steps under scoreboard control

- 2. Dispatch (Read operands)
  - When the instruction is issued, the execution unit is reserved (becomes *busy*)
  - Operands are read in the execution unit when they are both ready (i.e., are not results of still executing instructions). This prevents RAW hazards (this conservative approach was taken because the CDC 6600 was not pipelined)
- 3. Execution
  - One or more cycles depending on functional unit latency
  - When execution completes, the unit notifies the scoreboard it's ready to write the result

Execution steps under scoreboard control (c'ed)

- 4. Write result
  - Before writing, check for WAR hazards. If one exists, the unit is stalled until all WAR hazards are cleared (note that an instruction in progress, i.e., whose operands have been read, won't cause a WAR)
- 5. Delay (you can forget about this one)
  - Because forwarding is not implemented, there should be one unit of delay between writing and reading the same register (this restriction seems artificial to me and is "historical").
  - Similarly, it takes one unit of time between the release of a unit and its possible next occupancy

Optimizations and Simplifications

- There are opportunities for optimization such as:
  - Forwarding
  - Buffering for one copy of source operands in execution units (this allows reading of operands one at a time and minimizing the WAR hazards)
- We have assumed that there could be concurrent updates to (different) registers.
  - Can be solved (dynamically) by grouping execution units together and preventing concurrent writes in the same group or by having multiple write ports in the register file (expensive but common nowadays)

What is needed in the scoreboard (slightly redundant info)

- Status of each functional unit
  - Free or busy
  - Operation to be performed
  - The names of the result *Fi* and source *Fj, Fk* registers
  - Flags *Rj, Rk* indicating whether the source registers are ready
  - Names *Qj,Qk* of the units (if any) producing values for *Fj, Fk*
- Status of result registers
  - For each *Fi* the name of the unit (if any), say *Pi* that will produce its contents (redundant but easy to check)
- The instruction status
  - Been issued, dispatched, in execution, ready to write, finished?

Condition checking and scoreboard setting

- Issue step
  - Unit free, say *Ua* and no WAW
- Dispatch (Read operand)step
  - *Rj* and *Rk* must be *yes* (results ready)
- Execution step
  - At end ask for writing permission (no WAR)
- Write result
  - Check if *Pi* is an *Fj, Fk(Rj , Rk= no)* in preceding instrs. If yes stall.

- Issue step
  - *Ua* busy and record *Fi,Fj,Fk*
  - Record *Qj, Qk* and *Rj,Rk*
  - Record *Pi = Ua*
- Dispatch (Read operand) step
- Execution step
- Write result
  - For subsequent instrs, if *Qj(Qk) = Ua, set Rj(Rk) to yes*
  - *Ua* free and *Pi = 0*

Example

Load F6, 34(r2)	Load f-p register F6
Load F2, 45(r3)	Load latency 1 cycle
MulF F0,F2,F4	Mult latency 10 cycles
Sub F8, F6,F2	Add/sub latency 2 cycles
DivF F10,F0,F6	Divide latency 40 cycles
Add F6,F8,F2	

→ RAW

- - - ▸ WAR

Assume that the 2 Loads have been issued, the first one completed, the second ready to write. The next 3 instructions have been issued (but not dispatched).

Instruction	Issue	Dispatch	Executed	Result written
Load F6, 34(r2)	yes	yes	yes	yes
Load F2, 45(r3)	yes	yes	yes	
Mul F0, F2, F4	yes			
Sub F8, F6, F2	yes			
Div F10, F0, F6	yes			
Add F6,F8,F2				

Functional Unit status

No	Name	Busy	Fi	Fj	Fk	Qj	Qk	Rj	Rk
1	Int	yes	F2	r3					
2	Mul	yes	F0	F2	F4	1		No	Y
3	Mul	no							
4	Add	yes	F8	F6	F2		1	Y	No
5	Div	yes	F10	F0	F6	2		No	Y

Register result status

F0 (2) F2 (1) F4 ( ) F6( ) F8 (4) F10 (5) F12 ...

Instruction	Issue	Dispatch	Executed	Result written
Load F6, 34(r2)	yes	yes	yes	yes
Load F2, 45(r3)	yes	yes	yes	yes
Mul F0, F2, F4	yes	yes		
Sub F8, F6, F2	yes	yes		
Div F10, F0, F6	yes			
Add F6,F8,F2				

1 cycle after 2nd load has written its result

Functional Unit status

No	Name	Busy	Fi	Fj	Fk	Qj	Qk	Rj	Rk
1	Int	no							
2	Mul	yes	F0	F2	F4			Y	Y
3	Mul	no							
4	Add	yes	F8	F6	F2			Y	Y
5	Div	yes	F10	F0	F6	2		No	Y

Register result status

F0 (2) F2 ( ) F4 ( ) F6( ) F8 (4) F10 (5) F12 ...

Instruction	Issue	Dispatch	Executed	Result written
Load F6, 34(r2)	yes	yes	yes	yes
Load F2, 45(r3)	yes	yes	yes	yes
Mul F0, F2, F4	yes	yes	in progress	
Sub F8, F6, F2	yes	yes	yes	yes
Div F10, F0, F6	yes			
Add F6,F8,F2	yes	yes	yes	

6 cycles later; Mul in execution; Sub has completed;Div issues; Add waits for writing

Functional Unit status

No	Name	Busy	Fi	Fj	Fk	Qj	Qk	Rj	Rk
1	Int	no							
2	Mul	yes	F0	F2	F4			Y	Y
3	Mul	no							
4	Add	yes	F6	F8	F2			Y	Y
5	Div	yes	F10	F0	F6	2		No	Y

Register result status

F0 (2) F2 ( ) F4 ( ) F6(4) F8 ( ) F10 (5) F12 ...

Instruction	Issue	Dispatch	Executed	Result written
Load F6, 34(r2)	yes	yes	yes	yes
Load F2, 45(r3)	yes	yes	yes	yes
Mul F0, F2, F4	yes	yes	yes	yes
Sub F8, F6, F2	yes	yes	yes	yes
Div F10, F0, F6	yes	yes		
Add F6,F8,F2	yes	yes	yes	

4 cycles later (I think!) Mul is finished; Div can dispatch; Add will write at next cycle

Functional Unit status

No	Name	Busy	Fi	Fj	Fk	Qj	Qk	Rj	Rk
1	Int	no							
2	Mul	no							
3	Mul	no							
4	Add	yes	F6	F8	F2			Y	Y
5	Div	yes	F10	F0	F6			Y	Y

Register result status

F0 ( ) F2 ( ) F4 ( ) F6(4) F8 ( ) F10 (5) F12 ...

Instruction	Issue	Dispatch	Executed	Result written
Load F6, 34(r2)	yes	yes	yes	yes
Load F2, 45(r3)	yes	yes	yes	yes
Mul F0, F2, F4	yes	yes	yes	yes
Sub F8, F6, F2	yes	yes	yes	yes
Div F10, F0, F6	yes	yes		
Add F6,F8,F2	yes	yes	yes	yes

1 cycle later. Only Div is not finished

Functional Unit status

No	Name	Busy	Fi	Fj	Fk	Qj	Qk	Rj	Rk
1	Int	no							
2	Mul	no							
3	Mul	no							
4	Add	no							
5	Div	yes	F10	F0	F6			Y	Y

Register result status

F0 ( ) F2 ( ) F4 ( ) F6( ) F8 ( ) F10 (5) F12 ...

UNIVERSITI SAINS MALAYSIA
SCHOOL OF COMPUTER SCIENCES

CDS504 – ENABLING TECHNOLOGIES & INFRASTRUCTURES FOR BIG DATA

Semester 1, Session 2018/2019

Test 2

17 December 2018

This is a closed-book test. Answer all six (6) questions. *Time: 1 hour*

1) For the following questions, select only one (1) correct answer from the square brackets.

   a) Most devices in a smart home are likely to be connected using [*mesh / star / tree / ring / bus*] topology.

   b) A company that uses Microsoft Excel to analyze customer purchase patterns is an example of [*horizontal/ vertical / diagonal / precision*] application.

   c) Internet of Things (IoT) is a [*superset / subset / empty set / disjointed set*] of Internet of Everything (IoE).

   d) If two of the six packets received are corrupted, it is a case of [*packet loss / packet error / network latency / network jitter*]

   e) If only two of the six packets sent are received, it is a case of [*packet loss / packet error / network latency / network jitter*].

   f) Network jitter is important for [*Facebook updates / Skype video call / WhatsApp messages / browsing news websites*]

   g) [*DiffServ / HTTPS / DNS / SNMP / DHCP*] protocol ensures a minimum level of performance for a network.

   h) Congestion in a network can be handled using [*UDP / TCP / IPv4 / IPv6*] packets.

   i) In a congested network, it is advisable not to use [*multipathing / multihop path splitting / parallel streams*] to transfer files.

   j) [*DCell / Torus 2D / Torus 3D*] topology requires the least number of connections.

(20/100)

2) Explain how the following use big data networks. Use appropriate example to justify your answer.
   a) Smart home
   b) Wearable devices.

(20/100)

3) Most networks, including internet, are based on IP (internet protocol).
   a) Explain the statement "*IP networks are connection-less with best effort delivery.*"
   b) What could be the possible problems faced if a company's big data network is run on basic IP-based internet?
   c) Give a solution to one of the problems mentioned in 3(b).

(20/100)

4) The standard protocol to transfer files between nodes is FTP (file transfer protocol).
   a) Show how FTP works.
   b) It is said that FTP is not suitable for big data networks. Why? Give two reasons.
   c) What is suitable for transferring files on big data networks?

(15/100)

5) Three network survivability approaches have been suggested:
   - 1+1 Backup Protection,
   - 1:1 Backup Protection,
   - Shared Backup Protection.

   a) Which approach(es) allows some its nodes and links to go to sleep?
   b) Which approach(es) sends duplicated data on multiple links simultaneously?
   c) Which approach(es) can handle multiple primary path failures?
   d) Which approach has the least reserved bandwidth for its links?
   e) Which approach has the fastest recovery after failure?

(10/100)

6) Briefly explain the following.
   a) Torus topology.
   b) Equal Cost Multi Path (ECMP) routing protocol.
   c) Software Defined Network (SDN).

(15/100)

(a) In big data era, two of the key features are volume and velocity. State the goals and strategies in solving the challenges that associated with these features.

Dalam era data raya, dua ciri utama ialah isipadu dan halaju. Nyatakan matlamat dan strategi untuk menyelesaikan cabaran-cabaran yang terlibat dengan ciri-ciri ini.

(4/100)

(b) Hardware infrastructure consists of various kinds of processors.

Infrakstrutur perkasasan mengandungi pelbagai jenis pemproses.

(i) How can we combine these various types of processors to provide high performance computing platform for big data processing?

Bagaimana pelbagai jenis pemproses ini digabungkan untuk menyediakan pelantar pemprosesan tinggi untuk pemprosesan data raya?

(ii) Describe the type of middleware that is suitable for these kinds of processing.

Huraikan jenis "perisian tengah" yang sesuai untuk pemprosesan sebegini.

(6/100)

(c) Big data computing can be divided into two categories: one is the same program with different data set, whereas the other one is to mine any patterns using different artificial intelligent algorithms with huge variety of dataset.

Pengkomputeran data raya dapat dibahagikan kepada dua kategori: pertama program yang sama dengan set data berlainan, manakala yang satu lagi melombong sebarang corak dengan algoritma kecerdasaan buatan dengan set data besar pelbagai jenis.

(i) Show the details with some examples.

Tunjukkan perincian dengan contoh-contoh.

(ii) State the category that is more popular in grid computing.

Nyatakan kategori yang lebih terkenal dalam pengkomputeran grid.

(iii) In cloud computing how can you support both categories?

Dalam pengkomputeran awan bagaimana kita dapat menyokong kedua-dua kategori ini?

(9/100)

(d) In cloud computing, service oriented approach is used. Describe **three (3)** kinds of computational commodity as a service including infrastructure as a service, with main core technologies associated with each kind of service.

Dalam pengkomputeran awan, pendekatan berorientasi servis digunakan. Huraikan ***tiga (3)*** *jenis komoditi pengiraan sebagai servis termasuk infrastruktur sebagai servis dengan teknologi teras yang terlibat pada setiap jenis servis.*

(6/100)

(a) (i) State the objectives of optimization methods for scheduling in big data computing.

Nyatakan objektif-objektif bagi kaedah pengoptimuman untuk penjadualan dalam pengkomputeran data raya.

(ii) Describe advantages of the dynamic scheduling compared to static scheduling.

Huraikan kelebihan-kelebihan penjadualan dinamik berbanding penjadualan statik.

(5/100)

(b) Deep learning is one of the optimization. Propose a way to use high performance processing for this kind of optimization in big data.

Pembelajaran dalam merupakan sejenis pengoptimuman. Cadangkan satu cara menggunakan pemprosesan tinggi untuk pengoptimuman jenis ini dalam data raya.

(7/100)

(i) Distinguish the major characteristic between nosql with relational database.

Bezakan ciri utama antara nosql dengan pangkalan data hubungan.

(ii) Describe the main strength of Cassandra in CAP theorem.

Huraikan kelebihan utama Cassandra dalam teorem CAP.

(6/100)

In big data processing, Apache Hadoop and nosql database can be used for as infrastructure.

Dalam pemprosesan data raya, Apache Hadoop dan pangkalan data nosql boleh digunakan sebagai infrastruktur.

(i) Explain **one (1)** example to use these software tools to support big data processing.

*Terangkan **satu (1)** contoh untuk menggunakan alat perisian ini untuk menyokong pemprosesan data raya.*

(ii) Design your system and describe the activities using the above mentioned tools to support data analytics.

Reka bentuk sistem dan terangkan aktiviti menggunakan alat tersebut di atas untuk menyokong analisis data.

(iii) Show the example commands involved.

Tunjukkan arahan-arahan contoh yang terlibat.

(7/100)

Briefly describe **one (1)** well-known big data network that you know. Show how it is different from regular networks.

*Terangkan secara ringkas **satu (1)** rangkaian data raya terkenal yang anda ketahui. Tunjukkan bagaimana ia berbeza daripada rangkaian-rangkaian biasa.*

(5/100)

b) Discuss the following statements using appropriate examples.

Bincang kenyataan-kenyataan berikut, menggunakan contoh-contoh yang sesuai.

(i) "A big data network requires special topology."

"Suatu rangkaian data raya memerlukan topologi khas."

(ii) "A big data network requires special routing."

"Suatu rangkaian data raya memerlukan penghalaan khas."

(10/100)

(c) Green networking is a new trend adopted by many big data networks.

Perangkaian hijau adalah aliran baharu yang diguna pakai oleh banyak rangkaian data raya.

(i) What is green networking? Why is it important for big data networks?

Apa itu perangkaian hijau? Mengapa ia penting untuk rangkaian-rangkaian data raya?

(ii) Give **one (1)** example of how green networking can be used in a big data network.

*Berikan **satu (1)** contoh bagaimana perangkaian hijau boleh digunakan dalam rangkaian data raya.*

(iii) What are the possible obstacles faced when implementing green networking in big data networks?

Apakah kekangan-kekangan yang mungkin dihadapi apabila melaksanakan perangkaian hijau dalam rangkaian data raya?

(10/100)

a) Assume a company has deployed a host-based network intrusion detection system (NIDS) in its big data network, which uses signature-matching technique.

Anggap suatu syarikat telah melaksanakan sistem pengesanan pencerobоh rangkaian (NIDS) berasaskan hos pada rangkaian data rayanya, yang menggunakan teknik pemadanan-tandatangan.

(i) What types of security threats this NIDS can detect?

Apakah jenis-jenis ancaman keselamatan yang boleh dikesan oleh NIDS ini?

(ii) What types of security threats this NIDS cannot detect?

Apakah jenis-jenis ancaman keselamatan yang tidak boleh dikesan oleh NIDS ini?

(iii) What else can the company do to improve security of its big data network?

Apa lagi yang boleh dilakukan oleh syarikat ini untuk meningkatkan keselamatan rangkaian data rayanya?

Explain each answer. Provide appropriate examples.

Terangkan setiap jawapan. Berikan contoh-contoh yang sesuai.

(10/100)

efly describe how the following affects the performance of a big data twork.

rangkan secara ringkas bagaimana setiap yang berikut memberi esan terhadap pencapaian rangkaian data raya.

Vertical applications.

Aplikasi menegak.

i) Unstructered data.

Data tanpa sktrutur.

iii) Data transfer protocols.

Protokol-protokol pemindahan data.

(iv) Network protocols.

Protokol-protokol rangkaian.

(v) Network partitioning.

Pemetakan rangkaian.

(10/100)

Give **two (2)** advantages and **two (2)** disadvantages of using sampling in packet-based network intrusion detection system (NIDS) for big data networks.

*Berikan **dua (2)** kebaikan dan **dua (2)** keburukan penggunaan pensampelan dalam sistem pengesanan penceroboh rangkaian (NIDS) berasaskan paket untuk rangkaian data raya.*

(5/100)

- oooOooo -